# Protocolo TicAmazon – Protocolo interno, Isla 4

## Formato de mensaje

```
ORIGEN|DESTINO/TIPO_MENSAJE/campo1/campo2/...
```

**Significado de los separadores**

| Símbolo | Separa | Ejemplo |
|---|---|---|
| `\|` | ORIGEN de DESTINO | `CLI_04\|INT_04/...` |
| `/` | DESTINO de TIPO_MENSAJE, y cada campo entre sí | `INT_04\|CLI_04/11/<categoria>` |

**Identificadores**

- ID Cliente: `CLI_NN`
- ID Intermediario: `INT_NN` (NN = número de isla)
- ID Servidor de productos: `SERV_NN` (NN = número del servidor dentro de la isla, 01 a 99). En la etapa 2 era solo `SERV`; con dos o más servidores hace falta distinguirlos.

El protocolo no fija un tamaño máximo de mensaje: lo acotan los máximos de cada campo. Cada implementación define su propio buffer; si recibe o tiene que enviar un mensaje que no le cabe, responde 91 (ERR_SIZE) en vez de cortarlo.

## Campos y expresiones regulares

`origen` y `destino` pasan a 6 o 7 caracteres porque `SERV_01` mide 7. Los campos nuevos de la etapa 3 están en la tabla *Campos nuevos*, al final de esta sección.

**Campos**

| Campo | Tipo | Tam_Min | Tam_Max |
|---|---|---|---|
| origen | string | 6 | 7 chars |
| destino | string | 6 | 7 chars |
| tipo_mensaje | int | 2 | 2 chars |
| categoria | string | 3 | 20 chars |
| producto | string | 1 | 50 chars |
| precio | float | - | 8 chars (NNNNN.NN) |
| stock | int | 1 | 5 dígitos (0-99999) |
| count | int | 1 | 3 dígitos (0-999) |

**Expresiones regulares para validación**

| Campo | Regex |
|---|---|
| origen / destino | `^(CLI\|INT\|SERV)_[0-9]{2}$` |
| tipo_mensaje | `^[0-9]{2}$` |
| categoria | `^[a-zA-Z]{3,20}$` |
| producto | `^[a-zA-Z0-9\-]{1,50}$` |
| precio | `^[0-9]{1,6}(\.[0-9]{1,2})?$` |
| stock | `^[0-9]{1,5}$` |
| count | `^[0-9]{1,3}$` |

**Campos nuevos**

| Campo | Tam_Min | Tam_Max | Regex |
|---|---|---|---|
| nombre | 1 | 20 chars | `^[a-zA-Z0-9\-]{1,20}$` |
| ip | 7 | 15 chars | `^([0-9]{1,3}\.){3}[0-9]{1,3}$` |
| puerto | 1 | 5 dígitos | `^[0-9]{1,5}$` |
| lista_categorias | 3 | 195 chars | categorías separadas por coma, cada una con la regex de categoria |
| timestamp | 8 | 8 chars (HH:MM:SS) | `^[0-9]{2}:[0-9]{2}:[0-9]{2}$` |
| estado | 2 | 2 chars | `^(OK\|9[0-2])$` (OK o el código de error) |

Cliente, Intermediario y Servidor validan los campos que reciben antes de procesarlos. Si algún campo no cumple con regex o tamaños determinados, salta mensaje de error.

## Transporte y puertos

Todo el tráfico va por TCP, con una conexión por solicitud y la respuesta por esa misma conexión. No se usa UDP: el registro necesita confirmación (71) y las reservas no toleran pérdidas.

| Enlace | Transporte | Puerto | Quién escucha |
|---|---|---|---|
| Cliente → Intermediario (mensajes 10 al 42 y 73 al 78) | HTTP sobre TCP | 80 (443 si se usa SSL) | Intermediario |
| Servidor de productos → Intermediario (70) | TCP, protocolo propio | 9040 | Intermediario |

| Enlace | Transporte | Puerto | Quién escucha |
|---|---|---|---|
| Intermediario → Servidor de productos (10, 20 y 50) | TCP, protocolo propio | el `puerto` anunciado en el 70 (sugerido 9050) | Servidor de productos |

En la simulación (buzón System V) no hay IP ni puertos: el canal del buzón identifica quién le habla a quién. Los campos `ip` y `puerto` del registro viajan igual, con valores de relleno.

## a. Interacción Cliente – Intermediario (HTTP)

El cliente se comunica con el servidor intermedio mediante el protocolo HTTP, en el puerto TCP 80 (o 443 si se usa SSL). El cuerpo de las peticiones y respuestas HTTP transporta los mensajes definidos en este protocolo. Las consultas 73 al 78 también van por HTTP y se describen en la sección d.

| Código | Nombre | Formato | Emisor → Receptor | Descripción |
|---|---|---|---|---|
| 10 | REQUEST_CATEGORIES | `CLI_04\|INT_04/10` | Cliente → Intermediario | Pide la lista de categorías (todavía no sabe cuáles hay) |
| 11 | CATEGORY_LIST | `INT_04\|CLI_04/11/<lista>` | Intermediario → Cliente | Responde con las categorías disponibles |
| 20 | REQUEST_PRODUCTS | `CLI_04\|INT_04/20/<categoria>` | Cliente → Intermediario | Pide los productos de una categoría ya elegida |
| 22 | PRODUCT_LIST | `INT_04\|CLI_04/22/<count>/<lista>` | Intermediario → Cliente | Responde con los productos de esa categoría |
| 23 | PRODUCT_LIST_EMPTY | `INT_04\|CLI_04/23/<texto>` | Intermediario → Cliente | Avisa que la categoría no tiene productos |
| 30 | ADD_TO_CART | `CLI_04\|INT_04/30/<producto>/<cantidad>` | Cliente → Intermediario | Agrega un producto al carrito |
| 32 | CART_UPDATED | `INT_04\|CLI_04/32/<producto>/<precio>/<cantidad>` | Intermediario → Cliente | Confirma lo que realmente quedó en el carrito |
| 40 | REQUEST_FACTURA | `CLI_04\|INT_04/40` | Cliente → Intermediario | Solicita el cierre del pedido |
| 41 | FACTURA | `INT_04\|CLI_04/41/<detalle>/<total>` | Intermediario → Cliente | Responde con la factura proforma |

| Código | Nombre | Formato | Emisor → Receptor | Descripción |
|---|---|---|---|---|
| 42 | CART_EMPTY | `INT_04\|CLI_04/42/<texto>` | Intermediario → Cliente | Avisa que el carrito está vacío |

## b. Interacción Intermediario – Servidor de productos

Entre el servidor intermedio y cada servidor de productos se utiliza el mismo formato de mensajes, mediante un protocolo propio directo (sin HTTP) sobre TCP. `SERV_NN` es el servidor al que se le habla o que responde; con varios servidores registrados, la sección Enrutamiento dice a cuál va cada mensaje.

| Código | Nombre | Formato | Emisor → Receptor | Descripción |
|---|---|---|---|---|
| 10 | REQUEST_CATEGORIES | `INT_04\|SERV_NN/10` | Intermediario → Servidor | Solicita las categorías disponibles en ese servidor |
| 11 | CATEGORY_LIST | `SERV_NN\|INT_04/11/<lista>` | Servidor → Intermediario | Responde con las categorías |
| 20 | REQUEST_PRODUCTS | `INT_04\|SERV_NN/20/<categoria>` | Intermediario → Servidor | Solicita los productos de una categoría |
| 22 | PRODUCT_LIST | `SERV_NN\|INT_04/22/<count>/<lista>` | Servidor → Intermediario | Responde con los productos disponibles |
| 50 | RESERVE_STOCK | `INT_04\|SERV_NN/50/<producto>/<cantidad>` | Intermediario → Servidor | Reserva existencias para una orden en curso |
| 51 | RESERVED | `SERV_NN\|INT_04/51/<producto>/<precio>/<cantidad_reservada>` | Servidor → Intermediario | Confirma la reserva, con el precio y la cantidad realmente reservada (puede ser menor a la pedida si no había suficiente stock). Si el producto no existe en ese servidor responde precio 0.00 y cantidad 0 |

## c. Registro dinámico de servidores de productos (nuevo)

Un servidor de productos puede incorporarse en cualquier momento, sin reiniciar el intermediario: se registra con el mensaje 70 y publica sus categorías. Desde ese instante el intermediario lo incluye en la tabla de rutas. El registro va por TCP al puerto 9040 del intermediario.

| Código | Nombre | Formato | Emisor → Receptor | Descripción |
|---|---|---|---|---|
| 70 | REGISTER_SERVER | `SERV_NN\|INT_04/70/<nombre>/<ip>/<puerto>/<lista_categorias>` | Servidor → Intermediario | Registra el servidor y publica sus categorías. Si el `SERV_NN` ya estaba registrado, reemplaza sus datos y categorías |
| 71 | REGISTER_ACK | `INT_04\|SERV_NN/71/<count>` | Intermediario → Servidor | Confirma el registro. `count` es el número de categorías que quedan registradas para ese servidor |

Reglas:

- El intermediario valida los campos del 70 con las regex de la tabla. Si alguno falla responde 90 y no registra nada.
- Una misma categoría puede estar publicada por más de un servidor.
- Cada registro y reemplazo queda en la bitácora.

## d. Consultas al intermediario (nuevo)

El cliente puede preguntarle al intermediario qué servidores hay registrados, qué categorías atiende cada uno y qué eventos han ocurrido.

| Código | Nombre | Formato | Emisor → Receptor | Descripción |
|---|---|---|---|---|
| 73 | REQUEST_SERVERS | `CLI_04\|INT_04/73` | Cliente → Intermediario | Pide los servidores registrados |
| 74 | SERVER_LIST | `INT_04\|CLI_04/74/<count>/<lista>` | Intermediario → Cliente | Cada entrada es `SERV_NN,nombre,ip,puerto`, separadas por `;` |
| 75 | REQUEST_ROUTES | `CLI_04\|INT_04/75` | Cliente → Intermediario | Pide la tabla de rutas |
| 76 | ROUTE_TABLE | `INT_04\|CLI_04/76/<count>/<lista>` | Intermediario → Cliente | Cada entrada es `categoria,SERV_NN`, separadas por `;`. Una categoría en dos servidores aparece dos veces |
| 77 | REQUEST_LOG | `CLI_04\|INT_04/77/<n>` | Cliente → Intermediario | Pide los últimos n eventos de la bitácora del intermediario |

| Código | Nombre | Formato | Emisor → Receptor | Descripción |
|---|---|---|---|---|
| 78 | LOG_LIST | `INT_04\|CLI_04/78/<count>/<lista>` | Intermediario → Cliente | Cada entrada es `timestamp,origen,destino,tipo,estado`, separadas por `;` |

Reglas:

- n es la cantidad de eventos que se piden (misma regex que count, de 0 a 999). La respuesta trae los n más recientes, del más nuevo al más antiguo, o todos si hay menos de n.
- Cada respuesta trae el count de entradas que incluye. Sin entradas responde count 0 y la lista vacía.
- Si las entradas no caben en el buffer del intermediario, se omiten las más antiguas hasta que quepan, y count refleja las que se enviaron. Si una respuesta de otro tipo (por ejemplo la tabla de rutas) no cabe, el intermediario responde 91 con el tipo que no pudo enviar.
- Servidores y rutas van ordenados por SERV_NN; las rutas de cada servidor, en el orden en que publicó sus categorías.
- Si n no cumple su regex, responde 90.

## Enrutamiento con varios servidores (nuevo)

El intermediario mantiene una tabla de servidores y una tabla de rutas (categoría y servidores que la atienden), que se llenan con el mensaje 70. Reglas por mensaje del cliente:

- **Categorías:** se comparan sin distinguir mayúsculas de minúsculas, tanto las del 70 como las del 20.
- **10:** envía el 10 a cada servidor registrado (en orden de `SERV_NN`), junta los 11 sin repetir categorías y responde un solo 11. Sin servidores registrados responde el 11 con la lista vacía.
- **20:** si ningún servidor atiende la categoría, responde 23 sin consultar a nadie. Si la atienden uno o más, envía el 20 a cada uno y responde un solo 22 con los productos juntos y `count` sumado.
- **30:** recuerda en qué servidor vio cada producto en los 22 (si está en varios, el de menor `SERV_NN`) y envía el 50 solo a ese. Si no lo recuerda, prueba el 50 en orden de `SERV_NN` y se queda con el primero que reserve más de 0. Si ninguno reserva, responde 32 con precio 0.00 y cantidad 0.
- **40:** igual que antes: usa el carrito que ya tiene y no consulta a ningún servidor.
- **Servidor que no responde:** tras 2 reintentos lo anota en la bitácora. En un 30 responde 92. En un 10 o un 20 responde con lo que contestaron los demás, y solo da 92 (con el primer servidor que falló) si no contestó ninguno.

## Bitácora de eventos (nuevo)

Cada componente registra todos los eventos que envía o recibe: mensajes y errores 90 al 92, registros y reemplazos de servidores (70), consultas 73, 75 y 77, y fallos de comunicación. La bitácora del intermediario se consulta en tiempo real con el mensaje 77; el intermediario anota la consulta después de responderla, así que la respuesta no incluye la consulta que la pidió.

Formato de cada evento:

| Campo | Contenido |
|---|---|
| timestamp | Hora local, `HH:MM:SS` |
| origen | Emisor del mensaje |
| destino | Receptor del mensaje |
| tipo | `tipo_mensaje`, 2 dígitos |
| estado | `OK` o el código de error (90 al 92) |
| detalle | Texto libre. Solo existe en la bitácora local, no viaja en el 78 |

En la terminal la simulación imprime cada mensaje una sola vez, con el formato [origen -> destino] y una explicación. El receptor lo confirma con una línea ACK en sentido contrario. El ACK es solo una confirmación en pantalla: no es un mensaje del protocolo ni viaja por la red.

```
[CLI_04  -> INT_04 ] CLI_04|INT_04/30/Flan-caramelo/2   solicitud enviada
[INT_04  -> CLI_04 ] ACK                                 peticion recibida
```

## Distribución de red

| Red | Rango | VLAN |
|---|---|---|
| Red pública | 10.1.35.0/24 | — |
| Isla 4 | 172.16.123.64/28 | VLAN 240 o 540 |
| Isla Profes | 172.16.123.0/28 | VLAN 200 o 500 |

## Escenarios de uso

Los escenarios 1 al 10 son los de la etapa 2, con `SERV` reemplazado por `SERV_NN`. Suponen dos servidores registrados: `SERV_01` (postres y caramelos) y `SERV_02` (bebidas y reposteria). Los escenarios 11 al 16 son nuevos.

### 1. Solicitud de categorías disponibles

```
CLI_04|INT_04/10
INT_04|SERV_01/10
SERV_01|INT_04/11/postres,caramelos
INT_04|SERV_02/10
SERV_02|INT_04/11/bebidas,reposteria
INT_04|CLI_04/11/postres,caramelos,bebidas,reposteria
```

### 2. Solicitud del listado de productos de una categoría

```
CLI_04|INT_04/20/postres
INT_04|SERV_01/20/postres
SERV_01|INT_04/22/3/Flan-caramelo,3.20,20;Tres-leches,3.80,15;Tiramisu,4.50,10
INT_04|CLI_04/22/3/Flan-caramelo,3.20,20;Tres-leches,3.80,15;Tiramisu,4.50,10
```

### 3. Si la categoría no tiene productos

El servidor atiende la categoría pero no tiene productos en ella:

```
SERV_01|INT_04/22/0
INT_04|CLI_04/23/No hay productos disponibles en la categoria "postres"
```

Si ningún servidor registrado atiende la categoría, el intermediario responde 23 sin consultar a nadie:

```
CLI_04|INT_04/20/licores
INT_04|CLI_04/23/No hay productos disponibles en la categoria "licores"
```

### 4. Agregar producto al carrito

```
CLI_04|INT_04/30/Flan-caramelo/2
INT_04|SERV_01/50/Flan-caramelo/2
SERV_01|INT_04/51/Flan-caramelo/3.20/2
INT_04|CLI_04/32/Flan-caramelo/3.20/2
```

### 5. Si no hay stock suficiente para toda la cantidad pedida

El servidor reserva lo que sí tiene disponible y responde con esa cantidad, no con un mensaje de error. El intermediario compara la cantidad pedida contra la confirmada para saber que fue parcial.

```
CLI_04|INT_04/30/Flan-caramelo/50
INT_04|SERV_01/50/Flan-caramelo/50
SERV_01|INT_04/51/Flan-caramelo/3.20/20
INT_04|CLI_04/32/Flan-caramelo/3.20/20
```

### 6. Solicitud de factura proforma

```
CLI_04|INT_04/40/1/Flan-caramelo,2
INT_04|CLI_04/41/6.40/1/Flan-caramelo,2,6.40
```

El intermediario arma la factura con los productos ya confirmados en pasos anteriores de 30/ADD_TO_CART (ya tiene precio y cantidad validados), no vuelve a consultar a ningún servidor.

### 7. Si el carrito está vacío

```
CLI_04|INT_04/40/0
INT_04|CLI_04/42/El carrito esta vacio
```

### 8. Error de formato: no cumple regex

```
DETECTOR|SENDER/90/<tipo_mensaje_original>/<campo_invalido>/<valor_recibido>
```

### 9. Error de tamaño

```
DETECTOR|SENDER/91/<tipo_mensaje_original>/<campo>/<tamano_recibido>/<tamano_maximo>
```

### 10. Fallo de comunicación

```
DETECTOR|SENDER/92/<componente_no_responde>/<tipo_mensaje_pendiente>/<intentos>
```

### 11. Registro de un servidor e incorporación con el sistema en marcha

Antes del registro nadie atiende la categoría `bebidas`. Después del registro de `SERV_02`, el mismo pedido funciona sin reiniciar nada:

```
CLI_04|INT_04/20/bebidas
INT_04|CLI_04/23/No hay productos disponibles en la categoria "bebidas"
SERV_02|INT_04/70/Bodega2/172.16.123.69/9050/bebidas,reposteria
INT_04|SERV_02/71/2
CLI_04|INT_04/20/bebidas
INT_04|SERV_02/20/bebidas
SERV_02|INT_04/22/2/Cafe-frio,1.50,30;Te-helado,1.20,25
INT_04|CLI_04/22/2/Cafe-frio,1.50,30;Te-helado,1.20,25
```

### 12. Consulta de servidores registrados

```
CLI_04|INT_04/73
INT_04|CLI_04/74/2/SERV_01,Bodega1,172.16.123.68,9050;SERV_02,Bodega2,172.16.123.69,9
050
```

### 13. Consulta de la tabla de rutas

```
CLI_04|INT_04/75
INT_04|CLI_04/76/4/postres,SERV_01;caramelos,SERV_01;bebidas,SERV_02;reposteria,SERV_
02
```

### 14. Consulta de la bitácora

Pide los 7 eventos más recientes. Con n = 0 la lista viene vacía:

```
CLI_04|INT_04/77/7
INT_04|CLI_04/78/7/14:35:02,INT_04,CLI_04,76,OK;14:35:02,CLI_04,INT_04,75,OK;14:34:50
,INT_04,CLI_04,74,OK;14:34:50,CLI_04,INT_04,73,OK;14:33:11,INT_04,CLI_04,22,OK;14:33:
11,SERV_02,INT_04,22,OK;14:33:11,INT_04,SERV_02,20,OK
CLI_04|INT_04/77/0
INT_04|CLI_04/78/0/
```

### 15. Errores de registro

Categoría que no cumple la regex:

```
SERV_03|INT_04/70/Bodega3/172.16.123.70/9050/pos#res
INT_04|SERV_03/90/70/categoria/pos#res
```

### 16. Un servidor no responde

En un 30 el cliente recibe el error (si en un 10 o un 20 falla solo un servidor, el intermediario responde con los demás; ver Enrutamiento):

```
CLI_04|INT_04/30/Cafe-frio/2
INT_04|SERV_02/50/Cafe-frio/2
SERV_02|INT_04/(sin respuesta despues de 2 reintentos)
INT_04|CLI_04/92/SERV_02/50/2
```

## Ejemplo de salida completo

Un cliente que entra por primera vez, ve las categorías, elige una, agrega un producto al carrito y factura, con errores de ejemplo. Es el ejemplo de la etapa 2, con un único servidor `SERV_01` que atiende las cuatro categorías.

```
CLI_04|INT_04/10
INT_04|SERV_01/10
SERV_01|INT_04/11/postres,reposteria,caramelos,bebidas
INT_04|CLI_04/11/postres,reposteria,caramelos,bebidas

CLI_04|INT_04/20/"torta#3"
INT_04|CLI_04/90/20/categoria/"torta#3"

CLI_04|INT_04/20/postres
INT_04|SERV_01/20/postres
SERV_01|INT_04/22/3/Flan-caramelo,3.20,20;Tres-leches,3.80,15;Tiramisu,4.50,10
INT_04|CLI_04/22/3/Flan-caramelo,3.20,20;Tres-leches,3.80,15;Tiramisu,4.50,10

CLI_04|INT_04/30/CafeFrio/2
INT_04|SERV_01/50/CafeFrio/2
SERV_01|INT_04/(sin respuesta despues de 2 reintentos)
INT_04|CLI_04/92/SERV_01/50/2

CLI_04|INT_04/30/Flan-caramelo/2
INT_04|SERV_01/50/Flan-caramelo/2
SERV_01|INT_04/51/Flan-caramelo/3.20/2
INT_04|CLI_04/32/Flan-caramelo/3.20/2

CLI_04|INT_04/40/1/Flan-caramelo,2
INT_04|CLI_04/41/6.40/1/Flan-caramelo,2,6.40
```